

Neural Style Transfer: Report on Implementation and Findings

Aditi More

011937056

Implementation Summary

This assignment involved implementing neural style transfer using a pre-trained VGG19 convolutional neural network as described in the paper *"Image Style Transfer Using Convolutional Neural Networks"* by Gatys et al.

The following tasks were completed step-by-step:

1. **Model Setup:**

The features portion of VGG19 was loaded and frozen to prevent weight updates. This ensured only the target image was optimized during training.

2. **Image Loading:**

3. A helper function was used to load and transform both content and style images into normalized tensors. The style image was resized to match the content image's dimensions.

4. **Feature Extraction:**

The `get_features()` function was implemented to extract features from intermediate VGG layers. The following layers were used:

- Content: conv4_2
- Style: conv1_1, conv2_1, conv3_1, conv4_1, conv5_1

5. **Gram Matrix:**

The `gram_matrix()` function was written to capture style by computing correlations between feature maps.

6. **Loss Calculation:**

- **Content loss** was computed as the MSE between the target and content image at conv4_2.
- **Style loss** was calculated using the MSE between the Gram matrices of target and style images, with layer-specific weights.
- **Total loss** combined both with adjustable content and style weights (α and β).

7. **Optimization Loop:**

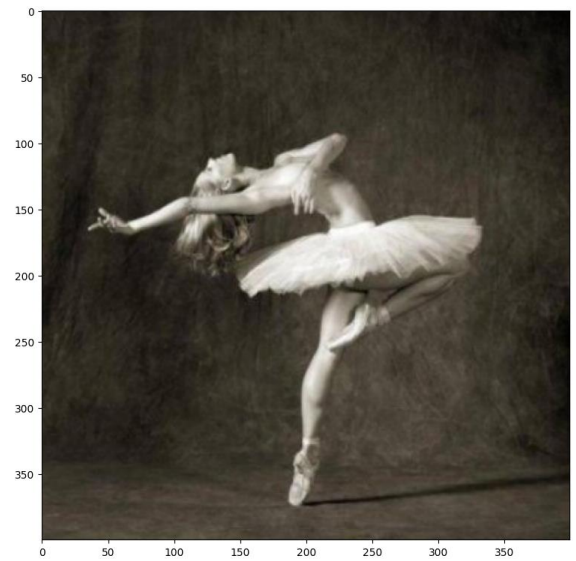
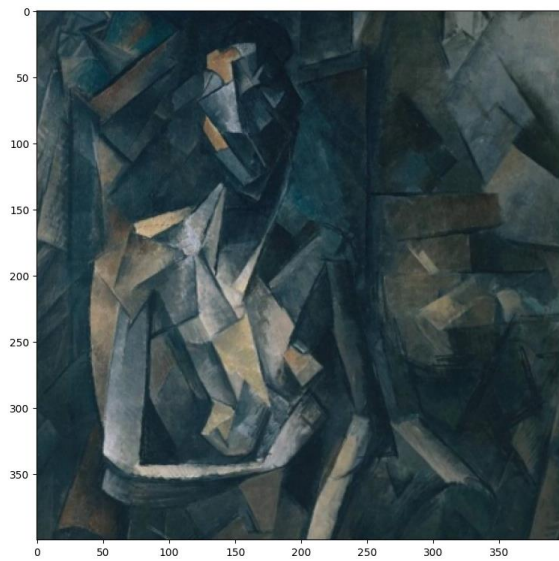
The target image (a clone of the content image) was optimized using the Adam optimizer. Training was run over several iterations with periodic display of the output image.

Experiments

I conducted many experiments by varying the content/style image pairs and hyperparameters such as learning rate, style weight, and number of training steps. I am showing two of those here.

Experiment 1

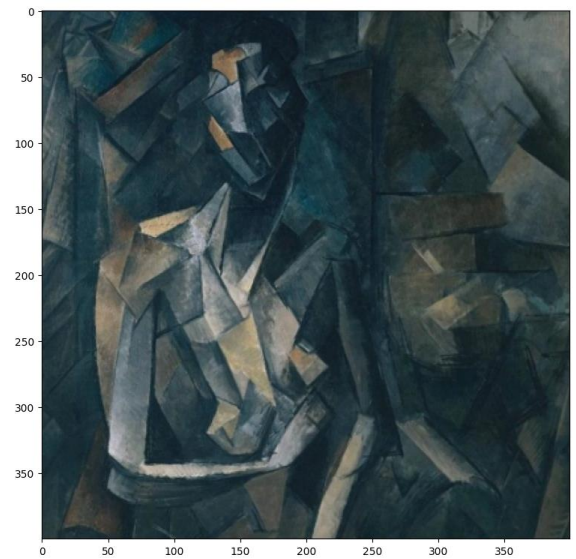
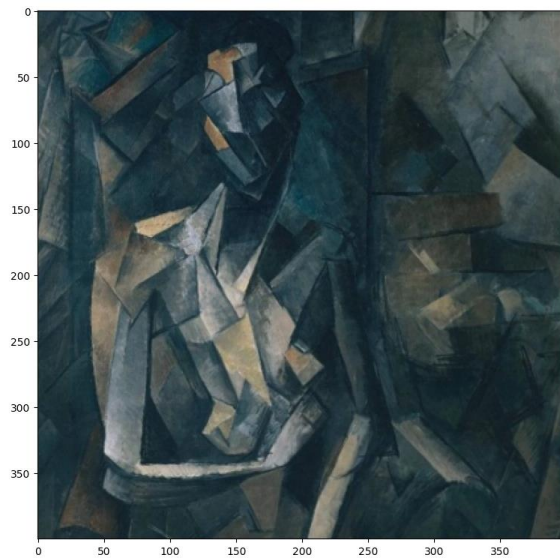
- **Content Image:** Picasso (from PyTorch tutorial)
- **Style Image:** Dancing (from PyTorch tutorial)



- **Hyperparameters:**
 - style_weight = 1e6
 - content_weight = 1
 - learning_rate = 0.003
 - steps = 2000

Observation:

- The final image appeared visually very close to the original content image.
- Even after 2000 iterations, the style effect was subtle.



Code:

```
# for displaying the target image, intermittently
```

```
show_every = 400
```

```
# iteration hyperparameters
```

```
optimizer = optim.Adam([target], lr=0.003)
```

```
steps = 2000 # decide how many iterations to update your image (5000)
```

```
for ii in range(1, steps + 1):
```

```
    # Get features from your target image
```

```
    target_features = get_features(target, vgg)
```

```
    # Calculate the content loss
```

```
    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])  
** 2)
```

```
    # Initialize the style loss
```

```
    style_loss = 0
```

```
    for layer in style_weights:
```

```
# Target style feature
target_feature = target_features[layer]
_, d, h, w = target_feature.shape

# Gram matrices
target_gram = gram_matrix(target_feature)
style_gram = style_grams[layer]

# Style loss for this layer
layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram) ** 2)

# Normalize and add to total style loss
style_loss += layer_style_loss / (d * h * w)

# Calculate total loss
total_loss = content_weight * content_loss + style_weight * style_loss

# Update your target image
optimizer.zero_grad()
total_loss.backward()
optimizer.step()

# Display intermediate images and print the loss
if ii % show_every == 0:
    print('Total loss: ', total_loss.item())
    plt.imshow(im_convert(target))
    plt.title(f'Step {ii}')
    plt.show()
```

- I also tried
 - style_weight = 1e6

- `content_weight = 1`
- `learning_rate = 0.003`
- `steps = 3000`

for the same pictures and I also tried to change the steps to 4000 keeping other parameters same but it did not produce drastically different results.

Experiment 2

- **Content Image:** A vibrant still from the movie *Luca*
- **Style Image:** *Starry Night* by Van Gogh



- **Hyperparameters:**
 - `style_weight = 2e7`
 - `content_weight = 1`
 - `learning_rate = 0.01`
 - `steps = 5000`

Observation:

- Higher values for both `style_weight` and `learning_rate` were used to increase the effect of the style.
- The content image had bold, colorful, cartoon-like visuals, while the style image had textured, brushy strokes.
- Despite this, the output remained visually very close to the content image.
- This may suggest that certain combinations of style/content images do not result in a strong visible transformation, or that the style loss was still being overpowered by the content structure.



Findings

- Neural style transfer is sensitive to image selection and hyperparameter tuning.
 - A higher style_weight and longer training steps did not always result in better stylization.
 - The learning rate also affects convergence: too low results in slow or no change, while too high may cause instability.
-

Final Choice: Experiment 2

I am choosing to submit **Experiment 2 (Luca + Starry Night)** as the result because:

- It demonstrates that I attempted more aggressive hyperparameter tuning (style_weight = $2e7$, lr = 0.01, steps = 5000) in line with the instructions to experiment.
- Even though the stylization was not visually strong, it reflects a realistic learning process and shows the limitations of certain image pairs.
- It also emphasizes that style transfer is not just about implementation but also about critically analyzing results and learning from them.